

# 18-Python Sets and Frozen Sets Lecture

---

## Introduction

---

- The lecture covers sets and frozen sets as important data structures in Python's standard library, alongside lists, tuples, and dictionaries.

## Definition of Set and Frozen Set

---

- **Set:**  
A collection type and iterable that stores unique, unordered elements. Sets are mutable, meaning elements can be added or removed after creation.
- **Frozen Set:** Similar to sets but immutable. Once created, the content cannot be modified.

## Hashability and Storage

---

- Sets store only hashable (immutable) objects, leveraging Python's built-in hash function for element storage and retrieval.
- Mutable objects (like lists or dictionaries) are unhashable and cannot be elements of a set; immutable objects (like integers, strings, tuples) are hashable.
- The hash value is used internally for efficient storage—elements are accessed via their hash values rather than indices.

## Set Creation and Syntax

---

- Sets are created using curly braces `{}` with comma-separated values (e.g., `{1, 2, 3}`), but empty sets require the `set()` function; `{}` alone creates an empty dictionary.
- To convert an existing iterable (e.g., list or tuple) to a set, use `set(iterable)`. Only the keys of a dictionary are included if a dictionary is passed.
- Sets can be created via set comprehensions for compact, dynamic creation.

## Access and Membership

---

- Sets do not support indexing. Membership is checked using the `in` operator, which uses the hash value for rapid lookup.
- Iteration over a set is straightforward with a loop, as sets are iterable objects.

## Order and Uniqueness

---

- Sets are unordered; printing or iterating may yield elements in any order.
- All elements within a set are unique. Duplicate entries are automatically removed upon set creation.

## Practical Applications of Sets

---

- Useful for obtaining unique elements from a dataset, e.g., filtering repeated email addresses from a list.
- Efficient for checking membership and deduplication tasks.

# Set Operations and Methods

---

- **Membership & Iteration:** Use `in` to check presence; sets can be looped over.
- **Add:** `add(element)`  
adds a single hashable element. Adding a duplicate or unhashable element (e.g., list) has no effect or raises an error.
- **Clear:** `clear()` removes all elements, leaving the set empty.
- **Copy:** `copy()` returns a shallow copy of the set.
- **Pop:** `pop()` removes and returns a random element, as sets lack order.
- **Remove:** `remove(element)` deletes a specific element; raises error if absent.
- **Discard:** `discard(element)` removes the element if present; does nothing if absent.

## Set Mathematical Operations

---

- **Union:** `union(set2, set3, ...)` or `|`  
operator merges all elements from provided sets, duplicates omitted.
- **Intersection:** `intersection(set2, set3, ...)` or `&`  
operator returns elements common to all sets.
- **Difference:** `difference(set2)` or `-` operator gives elements in the first set but not in others.
- **Symmetric Difference:** `symmetric_difference(set2)` or `^`  
operator returns elements present in either set, but not both.

## Special Methods (Update variants)

---

- **Intersection Update:**  
Updates the set to contain only elements found in itself and the specified sets.
- **Difference Update:** Updates the set to remove elements found in others.
- **Symmetric Difference Update:** Updates the set to contain the symmetric difference with another.
- Update variants change the original set, while the base operations return a new set, leaving the originals unchanged.

## Subset & Superset

---

- **isSubset:** Returns True if all elements of one set are present in another.
- **isSuperset:** Returns True if the set contains all elements of another set.
- Real world analogy: TCS is a subset of Tata if every TCS employee is a Tata employee.

## Symmetric Difference & Disjoint

---

- **Symmetric Difference:**  
Elements in one set or the other but not both. Useful to identify non-overlapping elements.
- **isDisjoint:** Returns True if two sets have no elements in common (their intersection is empty).

## Frozen Set

---

- **Definition:** Created with `frozenset(iterable)`
  - . Holds unique, immutable items and is itself immutable.
- **Operations:**
  - Many set operations return new sets, but a frozen set's contents cannot be altered after creation.

## Conclusion & Next Steps

---

- Sets and frozen sets are powerful for handling unique, unordered collections and for various mathematical/data deduplication tasks.
- Familiarity with their methods supports efficient and readable Python code, especially in data analysis.
- The next lecture will introduce Python functions.
- Assignments related to set and dictionary have been extended and new assignments will be uploaded.

Notes powered by [CraftNote](#)